

Sprint 4 Structured Notes: Links, Paths, Addresses, and Navigation

Sprint 4 Goal

The goal of Sprint 4 is to learn how to connect HTML pages and resources.

Before Sprint 4, you learned how to write HTML elements, build a full HTML document, and mark text based on meaning.

In Sprint 4, you learn how to create links that connect your page to:

- another page in your project
- another website
- a file inside a folder
- a phone number
- an email address
- a navigation menu

By the end of this sprint, you should be able to:

- create links with the `a` element
- use the `href` attribute correctly
- write clear link text
- understand internal links and external links
- choose between absolute URLs and relative paths
- understand `/`, `./`, and `../` in file paths
- avoid local computer paths
- use the `target` attribute
- use `address` for contact information
- create phone links with `tel:`
- create email links with `mailto:`
- build a simple navigation area with `nav`
- recognize the five link states
- debug common beginner link mistakes

1. Why Sprint 4 Matters

A web page is usually not alone.

Most websites have many pages.

For example:

Home page
About page
Contact page
Resources page
Blog page

Links connect these pages together.

Without links, users would be stuck on one page.

Sprint 4 teaches you how to let users move from one place to another.

Simple idea:

HTML links create movement.

A link can move the user to:

- another page
- another website
- a file
- an email app
- a phone app
- another part of a site

2. The `a` Element

The `a` element creates a hyperlink.

The letter `a` means anchor.

Basic pattern:

```
<a href="destination">Clickable text</a>
```

Example:

```
<a href="about.html">About Page</a>
```

This creates a clickable link that goes to `about.html`.

The structure has three main parts:

```
<a href="about.html">About Page</a>
```

Part	Meaning
<code><a></code>	starts the anchor element
<code>href="about.html"</code>	tells the browser where the link goes
<code>About Page</code>	clickable text shown to the user
<code></code>	closes the anchor element

Simple mental model:

```
<a href="where-to-go">what-the-user-clicks</a>
```

Another example:

```
<a href="contact.html">Contact us</a>
```

Meaning:

When the user clicks "Contact us", go to `contact.html`.

3. The `href` Attribute

The `href` attribute gives the link destination.

Example:

```
<a href="resources.html">View resources</a>
```

Here:

```
href="resources.html"
```

means:

```
Go to the resources.html file.
```

If an anchor is meant to navigate somewhere, it needs `href`.

Weak example:

```
<a>Contact us</a>
```

Problem:

This has no destination.

It may look like an anchor in your code, but it does not tell the browser where to go.

Better:

```
<a href="contact.html">Contact us</a>
```

Rule:

```
If the link should take the user somewhere, use href.
```

4. What Part of a Link Is Clickable?

In this link:

```
<a href="about.html">About Page</a>
```

The clickable part is:

```
About Page
```

The user does not see `href="about.html"` on the page.

The user sees and clicks the text between the opening and closing anchor tags.

Bad link text:

```
<a href="about.html">Click here</a>
```

Better link text:

```
<a href="about.html">Read about our academy</a>
```

The clickable text should explain the destination or action.

5. Good Link Text

Good link text tells the user what will happen before they click.

Bad:

```
<a href="resources.html">Click here</a>
```

Why this is bad:

"Click here" does not explain where the link goes.

Better:

```
<a href="resources.html">View HTML learning resources</a>
```

This is better because the user knows the link goes to HTML resources.

Another weak example:

```
<a href="pricing.html">More info</a>
```

Better:

```
<a href="pricing.html">View course pricing details</a>
```

Good link text is important because:

- users can understand the link quickly
- the page is easier to scan
- screen reader users can understand links out of context
- repeated links like "click here" become confusing

Simple rule:

Do not write link text that only says "click here", "more", "details", or "read more" unless the surrounding context makes it very clear.

Better rule:

The link text should describe the destination.

6. External Links

An external link goes to another website.

Example:

```
<a href="https://www.freecodecamp.org">Visit freeCodeCamp</a>
```

This link goes outside your own project.

The destination is:

```
https://www.freecodecamp.org
```

This is called an absolute URL.

An absolute URL usually includes:

- protocol, such as `https://`
- domain name, such as `www.freecodecamp.org`
- optional path after the domain

Example:

```
<a href="https://example.com/articles/html-basics">Read an HTML basics article</a>
```

Use absolute URLs for external websites.

Simple rule:

```
External website = absolute URL
```

7. Internal Links

An internal link goes to another file inside your own project.

Example project:

```
project/  
  index.html  
  about.html  
  contact.html
```

If you are inside `index.html` and want to link to `about.html`, write:

```
<a href="about.html">About Page</a>
```

This is an internal link.

It does not need the full web address.

It only needs the path from the current file to the destination file.

This kind of path is called a relative path.

Simple rule:

```
Internal project file = relative path
```

8. Absolute URL vs Relative Path

Absolute URL

An absolute URL is a full web address.

Example:

```
<a href="https://www.freecodecamp.org">Visit freeCodeCamp</a>
```

Use absolute URLs for external websites.

Relative Path

A relative path points to a file based on the current file location.

Example:

```
<a href="about.html">About Page</a>
```

Use relative paths for files inside your own project.

Comparison

Type	Example	Use it for
Absolute URL	<code>https://www.freecodecamp.org</code>	external websites
Relative path	<code>about.html</code>	files in your project

Simple decision:

Is it another website?
Use an absolute URL.

Is it your own project file?
Use a relative path.

9. Understanding Project Folders

Many beginner path problems happen because the file structure is not clear.

Example project:

```
project/  
  index.html  
  about.html  
  contact.html  
  assets/  
    logo.svg  
    hero.jpg  
  pages/  
    resources.html
```

This means:

- `index.html`, `about.html`, and `contact.html` are in the main project folder
- `assets` is a folder inside the project
- `logo.svg` and `hero.jpg` are inside `assets`
- `pages` is another folder inside the project
- `resources.html` is inside `pages`

When writing paths, always ask:

```
Where am I now?  
Where do I need to go?
```

10. The / Symbol in Paths

The slash `/` separates folders and files in a path.

Example:

```

```

This means:

Go into the assets folder.
Find logo.png.

Another example:

```
<a href="pages/resources.html">View resources</a>
```

This means:

Go into the pages folder.
Find resources.html.

Simple rule:

/ separates folders and files.

11. The `./` Path

The symbol `./` means current folder.

Example:

```
<a href="./about.html">About Page</a>
```

This means:

Find about.html in the current folder.

This often works the same as:

```
<a href="about.html">About Page</a>
```

Both of these point to a file in the current folder.

Another example:

```
<link rel="stylesheet" href="./styles.css">
```

Meaning:

Find styles.css in the current folder.

Simple rule:

./ means current folder.

12. The `../` Path

The symbol `../` means parent folder.

The parent folder is one level above the current folder.

Example project:

```
project/  
  index.html  
  assets/  
    logo.svg  
  pages/  
    contact.html
```

Imagine you are writing code inside:

```
pages/contact.html
```

You want to use:

```
assets/logo.svg
```

But `assets` is not inside `pages`.

It is one level above `pages`.

So you write:

```

```

Breakdown:

```
../      go up one folder from pages/ to project/  
assets/  go into the assets folder  
logo.svg use the logo file
```

Simple rule:

```
../ means go up one folder.
```

Another example:

Project:

```
project/  
  pages/  
    contact.html  
    index.html
```

From `pages/contact.html`, link back to `index.html`:

```
<a href="../../index.html">Home</a>
```

Meaning:

```
Go up from pages/ to project/.  
Then find index.html.
```

13. Common Path Examples

Example 1: Same folder

Project:

```
project/  
  index.html  
  about.html
```

From `index.html` to `about.html`:

```
<a href="about.html">About Page</a>
```

or:

```
<a href="./about.html">About Page</a>
```

Example 2: File inside a child folder

Project:

```
project/  
  index.html  
  pages/  
    contact.html
```

From `index.html` to `pages/contact.html`:

```
<a href="pages/contact.html">Contact Page</a>
```

Example 3: File in parent folder

Project:

```
project/  
  index.html  
  pages/  
    contact.html
```

From `pages/contact.html` to `index.html`:

```
<a href="../../index.html">Home</a>
```

Example 4: File in sibling folder

Project:

```
project/  
  assets/  
    logo.svg  
  pages/  
    contact.html
```

From `pages/contact.html` to `assets/logo.svg`:

```

```

14. Do Not Use Local Computer Paths

A local computer path points to a file on your own machine.

Bad:

```
<a href="/Users/sangeeth/project/about.html">About</a>
```

This is bad because it only works on your computer.

If someone else opens the project, their computer probably does not have this path:

```
/Users/sangeeth/project/about.html
```

If you upload the project to the web, that local path will also fail.

Better:

```
<a href="about.html">About</a>
```

Or, if the file is in a folder:

```
<a href="./pages/about.html">About</a>
```

Simple rule:

Do not link to files using your computer's full local path.
Use project-relative paths instead.

15. The `target` Attribute

The `target` attribute controls where a link opens.

Example:

```
<a href="https://www.freecodecamp.org" target="_blank">Visit freeCodeCamp</a>
```

Here:

```
target="_blank"
```

means:

Open the link in a new tab or window.

Common `target` Values

Value	Meaning
<code>_self</code>	Open in the current tab or window. This is the default.
<code>_blank</code>	Open in a new tab or window.
<code>_parent</code>	Open in the parent frame. Mostly used with frames or iframes.
<code>_top</code>	Open in the top-level browsing context.
<code>_unfencedTop</code>	Experimental. Usually not needed for normal beginner HTML.

16. target="_self"

`_self` opens the link in the current tab or window.

This is the default behavior.

That means these two links usually behave the same:

```
<a href="about.html">About Page</a>
```

```
<a href="about.html" target="_self">About Page</a>
```

Use `_self` or omit `target` for normal internal navigation.

Example:

```
<a href="contact.html">Contact us</a>
```

This is usually better for pages inside your own website.

17. target="_blank"

`_blank` opens the link in a new tab or window.

Example:

```
<a href="https://developer.mozilla.org" target="_blank">Read MDN documentation</a>
```

This can be useful for external resources.

But do not use `_blank` automatically.

Use it only when opening a new tab helps the user.

Good possible use:


```
<a href="https://www.freecodecamp.org" target="_blank">Practice HTML on  
freeCodeCamp</a>
```

Usually unnecessary for internal links:

```
<a href="about.html" target="_blank">About us</a>
```

Better:

```
<a href="about.html">About us</a>
```

Simple rule:

Internal links usually stay in the same tab.
External reference links may open in a new tab if it helps the user.

18. `target="_parent"` and `target="_top"`

These are mostly relevant when pages are inside frames or iframes.

You do not need to master them deeply in Sprint 4.

`_parent`

`_parent` opens the link in the parent frame.

Example:

```
<a href="home.html" target="_parent">Open in parent frame</a>
```

`_top`

`_top` opens the link in the top-level browsing context.

Example:

```
<a href="home.html" target="_top">Open full page</a>
```

Beginner rule:

For normal pages, you will mostly use no target, `_self`, or `_blank`.

19. `_unfencedTop`

`_unfencedTop` is experimental.

For beginner HTML, you usually do not need it.

You only need to recognize that it exists as a target value mentioned in the curriculum.

Simple rule:

Do not use `_unfencedTop` in normal beginner projects.

20. Contact Information with `address`

The `address` element marks contact information.

Use it for contact details for:

- a person
- a company
- an organization
- the owner of a page
- the author of a section

Example:

```
<address>
  <h2>Contact Code Academy</h2>
  <p>
    123 Learning Street<br>
    Colombo<br>
    Sri Lanka
```

```
</p>
</address>
```

Here, the `address` element says:

```
This content is contact information.
```

Do not use `address` for any random location.

Use it when the location is contact information.

Bad use:

```
<p>The story happened at 123 River Road.</p>
```

This is just story content, not contact information.

Better use:

```
<address>
  <p>123 River Road<br>Colombo<br>Sri Lanka</p>
</address>
```

This is better only if it is actually a contact address.

21. Line Breaks with `br`

Physical addresses often need line breaks.

The `br` element creates a line break.

Example:

```
<p>
  123 Learning Street<br>
  Colombo<br>
  Sri Lanka
</p>
```

This displays the address on separate lines.

Important:

`br` is a void element.

That means it does not have a closing tag.

Correct:

```
<br>
```

Incorrect:

```
<br></br>
```

Use `br` when the line break is part of the meaning, such as in a postal address.

22. Phone Links with `tel:`

A phone link uses `tel:` inside `href`.

Example:

```
<a href="tel:+94112345678">+94 11 234 5678</a>
```

Meaning:

When supported, clicking this link can start a phone call.

Full example:

```
<p>Phone: <a href="tel:+94112345678">+94 11 234 5678</a></p>
```

Inside `href`, use a clean international format:

```
href="tel:+94112345678"
```

For visible text, you can use a more readable format:

```
+94 11 234 5678
```

Simple rule:

```
tel: is used for phone links.
```

23. Email Links with `mailto:`

An email link uses `mailto:` inside `href`.

Example:

```
<a href="mailto:contact@example.com">contact@example.com</a>
```

Meaning:

```
When supported, clicking this link can open the user's email app.
```

Full example:

```
<p>Email: <a href="mailto:contact@example.com">contact@example.com</a></p>
```

Simple rule:

```
mailto: is used for email links.
```

Important note:

`mailto:` can be useful, but it does not always work perfectly for every user.

Some users may not have an email app configured.

Some users may find it inconvenient if it opens an app unexpectedly.

Still, it is a valid HTML pattern for email contact links.

24. Full Contact Information Example

```
<address>
  <h2>Code Academy</h2>

  <p>
    123 Learning Street<br>
    Colombo<br>
    Sri Lanka
  </p>

  <p>
    Phone:
    <a href="tel:+94112345678">+94 11 234 5678</a>
  </p>

  <p>
    Email:
    <a href="mailto:contact@codeacademy.example">contact@codeacademy.example</a>
  </p>
</address>
```

This is good because:

- contact information is inside `address`
- the phone number uses `tel:`
- the email address uses `mailto:`
- the physical address uses `br` for line breaks
- the visible link text is clear

25. Navigation with `nav`

The `nav` element marks a navigation section.

A navigation section contains important links that help users move around the website.

Example:

```
<nav>
  <a href="index.html">Home</a>
  <a href="about.html">About</a>
  <a href="contact.html">Contact</a>
</nav>
```

This creates a simple navigation menu.

Each link should have a real `href`.

Bad:

```
<nav>
  <a>Home</a>
  <a href="click here">Click here</a>
</nav>
```

Problems:

- `Home` has no `href`
- `click here` is not a useful destination
- `Click here` is weak link text

Better:

```
<nav>
  <a href="index.html">Home</a>
  <a href="resources.html">View HTML resources</a>
</nav>
```

This is better because:

- every link has `href`
- link text explains where the user will go

26. Header Navigation Example

A common pattern is to put navigation inside a page header.

Example:

```
<header>
  <h1>Code Academy</h1>

  <nav>
    <a href="index.html">Home</a>
    <a href="about.html">About Code Academy</a>
    <a href="contact.html">Contact us</a>
  </nav>
</header>
```

Meaning:

- `header` introduces the page or site
- `h1` gives the main page heading
- `nav` groups the navigation links
- each `a` element gives one clickable destination

You will study semantic structure more deeply later, but this is a useful Sprint 4 pattern.

27. Link States

Links can have different states depending on user interaction.

The five link states are:

```
:link
:visited
:hover
:focus
:active
```

Recommended order:

```
link → visited → hover → focus → active
```

These are mostly used in CSS, but Sprint 4 wants you to recognize them.

`:link`

A normal unvisited link.

Meaning:

The user has not visited this link yet.

:visited

A link the user has already visited.

Meaning:

The browser knows the user already opened this destination.

:hover

A link when the mouse is over it.

Meaning:

The user is pointing at the link.

:focus

A link when it is selected by keyboard or another input method.

Meaning:

The link is currently focused.

This is important for keyboard users.

:active

A link while it is being activated.

Meaning:

The user is clicking or activating the link right now.

Link State Summary

State	Simple meaning
<code>:link</code>	normal unvisited link
<code>:visited</code>	already visited link
<code>:hover</code>	mouse is over the link
<code>:focus</code>	link is selected, often by keyboard
<code>:active</code>	link is being clicked or activated

Important beginner note:

You do not need to master CSS in Sprint 4.

Just understand that links can change appearance based on these states.

28. Complete Sprint 4 Example Page

Create a file called:

```
contact.html
```

Example code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Contact Page</title>
  <meta name="description" content="A beginner HTML contact page using links,
paths, phone links, and email links.">
</head>
<body>
  <header>
    <h1>Contact Code Academy</h1>

    <nav>
      <a href="index.html">Home</a>
      <a href="about.html">About Code Academy</a>
```

```
    <a href="resources.html">View HTML resources</a>
  </nav>
</header>

<main>
  <h2>Contact Details</h2>

  <address>
    <p>
      Code Academy<br>
      123 Learning Street<br>
      Colombo<br>
      Sri Lanka
    </p>

    <p>
      Phone:
      <a href="tel:+94112345678">+94 11 234 5678</a>
    </p>

    <p>
      Email:
      <a
href="mailto:contact@codeacademy.example">contact@codeacademy.example</a>
    </p>
  </address>

  <p>
    You can also
    <a href="resources.html">view our beginner HTML learning resources</a>.
  </p>

  <p>
    For external practice, visit
    <a href="https://www.freecodecamp.org" target="_blank">freeCodeCamp HTML
lessons</a>.
  </p>
</main>
</body>
</html>
```

29. Explanation of the Complete Example

Boilerplate

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Contact Page</title>
</head>
<body>
  ...
</body>
</html>
```

This comes from Sprint 2.

It creates a complete HTML document.

Header

```
<header>
  <h1>Contact Code Academy</h1>
  ...
</header>
```

The header introduces the page.

Navigation

```
<nav>
  <a href="index.html">Home</a>
  <a href="about.html">About Code Academy</a>
  <a href="resources.html">View HTML resources</a>
</nav>
```

This gives three internal navigation links.

Each link has:

- an `href`

- clear clickable text

Main Content

```
<main>
  <h2>Contact Details</h2>
  ...
</main>
```

The `main` element contains the main page content.

Contact Information

```
<address>
  ...
</address>
```

This marks the contact information.

Phone Link

```
<a href="tel:+94112345678">+94 11 234 5678</a>
```

This creates a phone link.

Email Link

```
<a href="mailto:contact@codeacademy.example">contact@codeacademy.example</a>
```

This creates an email link.

Internal Resource Link

```
<a href="resources.html">view our beginner HTML learning resources</a>
```

This goes to another page in the same project.

External Link

```
<a href="https://www.freecodecamp.org" target="_blank">freeCodeCamp HTML lessons</a>
```

This goes to another website and opens in a new tab.

30. Common Sprint 4 Mistakes and Fixes

Mistake 1: Anchor without href

Bad:

```
<a>Home</a>
```

Problem:

The link has no destination.

Better:

```
<a href="index.html">Home</a>
```

Mistake 2: Vague link text

Bad:

```
<a href="resources.html">Click here</a>
```

Problem:

The link text does not explain the destination.

Better:

```
<a href="resources.html">View HTML resources</a>
```

Mistake 3: Wrong href value

Bad:

```
<a href="click here">Click here</a>
```

Problem:

href should contain a real destination, not button-like text.

Better:

```
<a href="resources.html">View HTML resources</a>
```

Mistake 4: Local computer path

Bad:

```
<a href="/Users/sangeeth/project/about.html">About</a>
```

Problem:

This only works on one computer.

Better:

```
<a href="about.html">About</a>
```

Mistake 5: Using _blank for every link

Weak:

```
<a href="about.html" target="_blank">About us</a>
```

Problem:

Internal links usually should not open new tabs.

Better:

```
<a href="about.html">About us</a>
```

Mistake 6: Plain email text instead of email link

Weak:

```
<address>  
  Email: contact@example.com  
</address>
```

Better:

```
<address>  
  <p>Email: <a href="mailto:contact@example.com">contact@example.com</a></p>  
</address>
```

Mistake 7: Plain phone text instead of phone link

Weak:

```
<p>Phone: +94 11 234 5678</p>
```

Better:

```
<p>Phone: <a href="tel:+94112345678">+94 11 234 5678</a></p>
```

31. Debugging Sprint 4 Broken Code

Broken code:

```
<nav>  
  <a>Home</a>  
  <a href="click here">Click here</a>  
  <a href="/Users/sangeeth/project/about.html">About</a>  
</nav>  
  
<address>
```



```
Email: contact@example.com
</address>
```

Problems

Problem 1:

```
<a>Home</a>
```

This anchor has no `href`.

Problem 2:

```
<a href="click here">Click here</a>
```

The `href` value is not a real page path.

The link text is also vague.

Problem 3:

```
<a href="/Users/sangeeth/project/about.html">About</a>
```

This is a local computer path.

It should be a relative project path.

Problem 4:

```
<address>
  Email: contact@example.com
</address>
```

The email is plain text.

It should use a `mailto:` link.

Fixed Code

```
<nav>
  <a href="index.html">Home</a>
  <a href="resources.html">View HTML resources</a>
  <a href="about.html">About</a>
</nav>

<address>
  <p>
    Email:
    <a href="mailto:contact@example.com">contact@example.com</a>
  </p>
</address>
```

Why This Is Correct

- Home now has an href
- "Click here" was replaced with descriptive link text
- the local computer path was replaced with a relative path
- the email address now uses mailto:
- contact information remains inside address

32. Sprint 4 Practice Task

Create a file named:

```
resources.html
```

Build a resources page using only HTML.

Requirements

Your page should include:

1. A full Sprint 2 boilerplate
2. One h1
3. A nav with three internal links
4. Two external links using absolute URLs
5. Two internal links using relative paths
6. One link to ../assets/logo.svg

7. One `mailto:` link
 8. One `tel:` link
 9. Descriptive link text
 10. No local computer paths like `/Users/sangeeth/...`
-

33. Starter Practice Solution

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>HTML Resources</title>
  <meta name="description" content="A beginner HTML resources page using links,
paths, phone links, and email links.">
</head>
<body>
  <header>
    <h1>HTML Resources</h1>

    <nav>
      <a href="index.html">Home</a>
      <a href="about.html">About this HTML course</a>
      <a href="contact.html">Contact the course team</a>
    </nav>
  </header>

  <main>
    <h2>Internal Resources</h2>

    <p>
      <a href="lessons.html">View all HTML lessons</a>
    </p>

    <p>
      <a href="practice.html">Open HTML practice tasks</a>
    </p>

    <p>
      <a href="../assets/logo.svg">Open the course logo file</a>
    </p>

    <h2>External Resources</h2>

    <p>
```

```
    <a href="https://www.freecodecamp.org" target="_blank">Practice web
development on freeCodeCamp</a>
  </p>

  <p>
    <a href="https://developer.mozilla.org" target="_blank">Read web
documentation on MDN</a>
  </p>

  <h2>Contact</h2>

  <address>
    <p>
      Email:
      <a href="mailto:contact@example.com">contact@example.com</a>
    </p>

    <p>
      Phone:
      <a href="tel:+94112345678">+94 11 234 5678</a>
    </p>
  </address>
</main>
</body>
</html>
```

34. Sprint 4 Revision Questions

Use these questions to test yourself.

Question 1

What does the `a` element do?

Answer:

The `a` element creates a hyperlink.

Question 2

What does `href` do?

Answer:

The `href` attribute gives the destination of the link.

Question 3

What part of an anchor is clickable?

Answer:

The text or content between `<a>` and `</a>` is clickable.

Example:

```
<a href="about.html">About Page</a>
```

The clickable text is:

About Page

Question 4

When should you use an absolute URL?

Answer:

Use an absolute URL when linking to an external website.

Example:

```
<a href="https://www.freecodecamp.org">Visit freeCodeCamp</a>
```

Question 5

When should you use a relative path?

Answer:

Use a relative path when linking to a file inside your own project.

Example:

```
<a href="about.html">About Page</a>
```

Question 6

What does `./` mean?

Answer:

`./` means current folder.

Example:

```
<a href="./about.html">About Page</a>
```

Question 7

What does `../` mean?

Answer:

`../` means parent folder, or one folder up.

Example:

```
<a href="../index.html">Home</a>
```

Question 8

What does `target="_blank"` do?

Answer:

It opens the link in a new tab or window.

Question 9

What does `target="_self"` do?

Answer:

It opens the link in the current tab or window.

This is the default behavior.

Question 10

What does `tel:` do?

Answer:

`tel:` creates a phone link.

Example:

```
<a href="tel:+94112345678">+94 11 234 5678</a>
```

Question 11

What does `mailto:` do?

Answer:

`mailto:` creates an email link.

Example:

```
<a href="mailto:contact@example.com">contact@example.com</a>
```

Question 12

Why is “click here” weak link text?

Answer:

Because it does not explain where the link goes or what the user will get.

Better:

```
<a href="resources.html">View HTML learning resources</a>
```

Question 13

What are the five link states in order?

Answer:

```
:link  
:visited  
:hover  
:focus  
:active
```

Or:

```
link → visited → hover → focus → active
```

35. Sprint 4 Key Rules

Memorize these rules:

- Use `a` to create links.
 - Use `href` to give the link destination.
 - The text between `<a>` and `</a>` is clickable.
 - Use clear link text.
 - Avoid vague link text like “click here.”
 - Use absolute URLs for external websites.
 - Use relative paths for files inside your own project.
 - `/` separates folders and files.
 - `./` means current folder.
 - `../` means parent folder.
 - Do not use local computer paths like `/Users/...`.
 - `target="_self"` opens in the current tab.
 - `target="_blank"` opens in a new tab or window.
 - Do not use `_blank` automatically for every link.
 - Use `address` for contact information.
 - Use `tel:` for phone links.
 - Use `mailto:` for email links.
 - Use `nav` to group important navigation links.
 - Recognize the link states: `:link`, `:visited`, `:hover`, `:focus`, `:active`.
-

36. Sprint 4 Mini Glossary

a

The anchor element. It creates a hyperlink.

Example:

```
<a href="about.html">About</a>
```

href

The attribute that gives the link destination.

Example:

```
href="about.html"
```

Link text

The visible clickable text inside an anchor.

Example:

```
<a href="about.html">About Page</a>
```

The link text is:

```
About Page
```

Absolute URL

A full web address.

Example:

```
https://www.freecodecamp.org
```

Relative path

A path based on the current file location.

Example:

```
about.html
```

```
/
```

A folder separator.

Example:

```
assets/logo.svg
```

```
./
```

Current folder.

Example:

```
./about.html
```

```
../
```

Parent folder, or one folder up.

Example:

```
../index.html
```

```
target
```

An attribute that controls where the link opens.

Example:

```
<a href="https://example.com" target="_blank">Example</a>
```

`_self`

Opens the link in the current tab or window.

`_blank`

Opens the link in a new tab or window.

`address`

An element for contact information.

Example:

```
<address>  
  <p>Email: <a href="mailto:contact@example.com">contact@example.com</a></p>  
</address>
```

`tel:`

Creates a phone link.

Example:

```
<a href="tel:+94112345678">+94 11 234 5678</a>
```

`mailto:`

Creates an email link.

Example:

```
<a href="mailto:contact@example.com">contact@example.com</a>
```

`nav`

An element that groups navigation links.

Example:

```
<nav>
  <a href="index.html">Home</a>
  <a href="about.html">About</a>
</nav>
```

Link states

Different states a link can have during interaction.

Examples:

```
:link
:visited
:hover
:focus
:active
```

37. Sprint 4 Final Self-Check

You are ready to move on from Sprint 4 when you can do these without looking at notes:

- Write an anchor with `href`.
- Explain what `href` does.
- Write clear link text.
- Create an external link using an absolute URL.
- Create an internal link using a relative path.
- Explain the difference between absolute URL and relative path.
- Explain what `/` does in a path.
- Explain what `./` means.
- Explain what `../` means.
- Avoid local computer paths.
- Use `target="_blank"` correctly.
- Explain why `_blank` should not be used automatically.
- Create contact information with `address`.
- Create a phone link with `tel:`.
- Create an email link with `mailto:`.
- Build a simple `nav` section.
- List the five link states in order.
- Debug broken Sprint 4 link code.

38. Sprint 4 Completion Standard

Sprint 4 is complete when you can write links that are correct, readable, and useful.

You should be able to look at this broken code:

```
<nav>
  <a>Home</a>
  <a href="click here">Click here</a>
  <a href="/Users/sangeeth/project/about.html">About</a>
</nav>

<address>
  Email: contact@example.com
</address>
```

And fix it to:

```
<nav>
  <a href="index.html">Home</a>
  <a href="resources.html">View HTML resources</a>
  <a href="about.html">About</a>
</nav>

<address>
  <p>
    Email:
    <a href="mailto:contact@example.com">contact@example.com</a>
  </p>
</address>
```

You should also be able to explain why each fix is correct.

The final mental model is:

```
a = creates the link
href = where the link goes
link text = what the user understands and clicks
absolute URL = external website
relative path = file inside your project
/ = separates folders and files
./ = current folder
```

```
../ = parent folder  
tel: = phone link  
mailto: = email link  
nav = navigation links  
address = contact information
```

Main point of Sprint 4:

Write links that go to the right place and make sense before the user clicks them.